

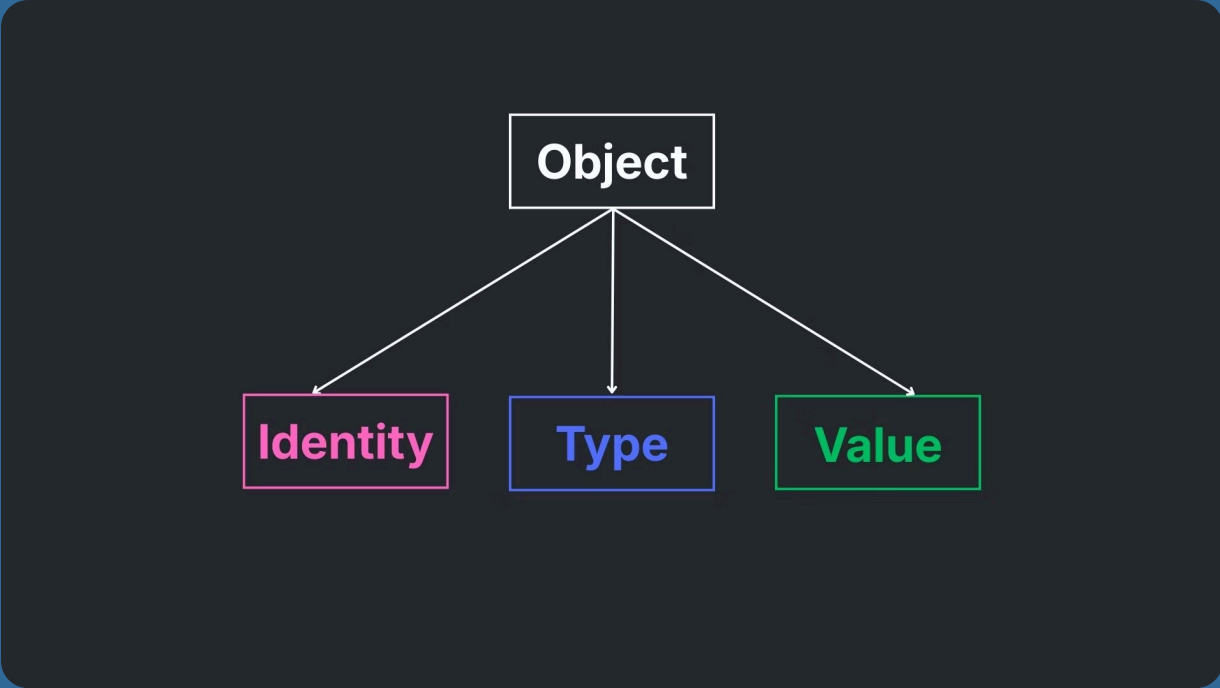
Mutable Datatypes

Lec-2

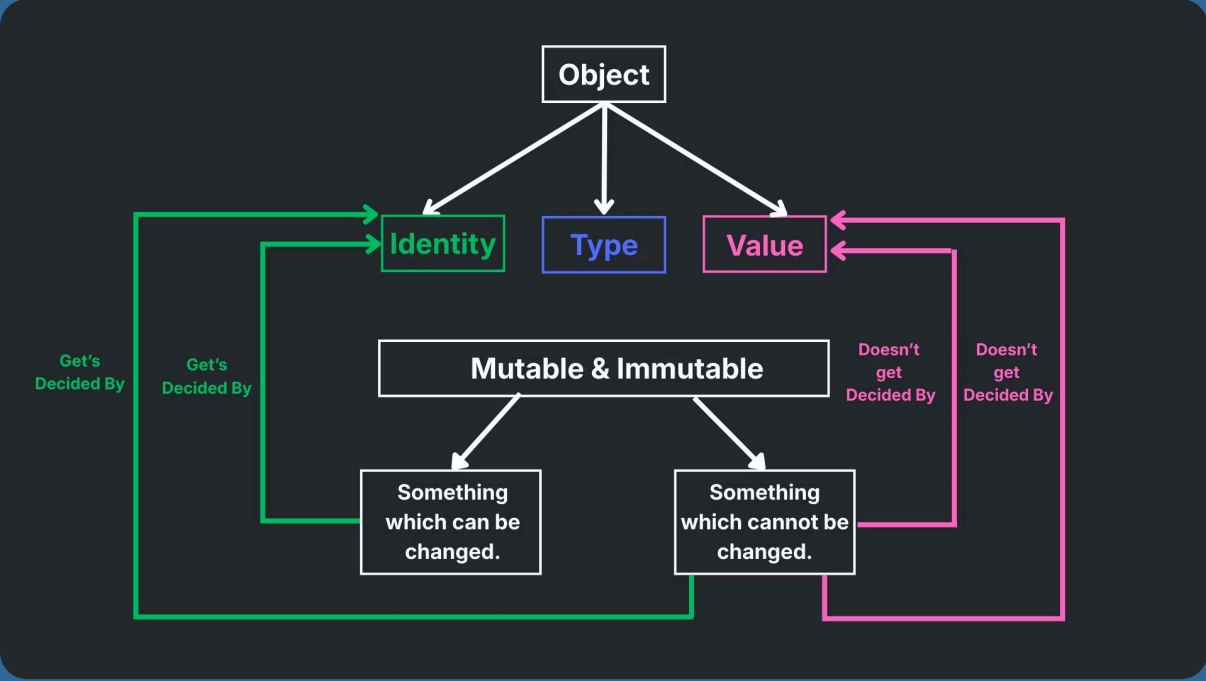
Mutable and Immutable

- Everything is Object in Python.
- Mutability means the ability of an object to get modified inside the memory after it's creation.
- Every Object has the following properties:

- Identity
- Type
- Value



- Always Remember it is the Identity of an object which helps us determine whether the object is Mutable or Immutable.



Immutable

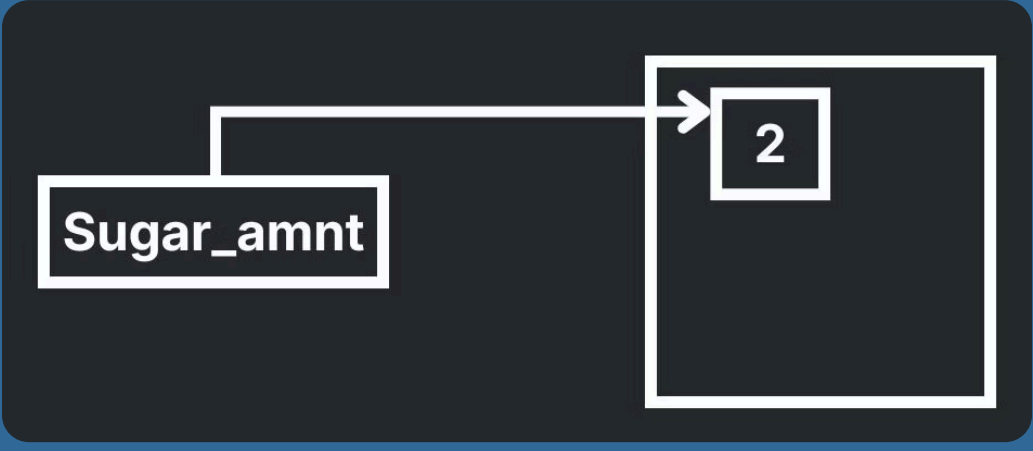
- Eg:

python:

```
sugar_amnt = 2
print(f'Initial Sugar: {sugar_amnt}')
```

output:

Initial Sugar: 2



python:

```
sugar_amnt = 2
print(f'Initial Sugar: {sugar_amnt}')
```

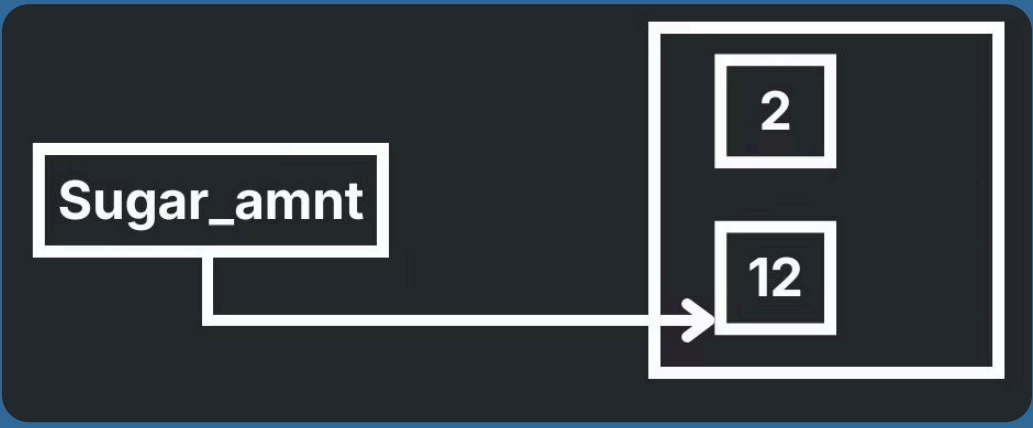


```
sugar_amnt = 12
print(f'Second Initial Sugar: {sugar_amnt}')
```

output:

Initial Sugar: 2

Second Initial Sugar: 12



- When we say Mutability doesn't get decided by value, we mean to say that when we assign different value to a same variable the reference to that variable inside the memory get's changed and the old value still remains inside the memory as unreferred.

python

```
print(f'Id of 2: {id(2)}')
```

output:

Id of 2: 140716054574232

python

```
print(f'Id of 12: {id(12)}')
```

output:

Id of 12: 140716054574552

- So the above object was Immutable i.e. numbers are immutable.

Mutable

- Eg:

python:

```
spice_mix = set()
print(spice_mix)
print(f'Initial spice mix id: {id(spice_mix)}')
```

output:

set()

Initial spice mix id: 2139208714496



python:

```
spice_mix.add("Cardamom")
print(spice_mix)
print(f'After spice mix id: {id(spice_mix)}')
```

output:

{'Cardamom'}

After spice mix id: 2139208714496



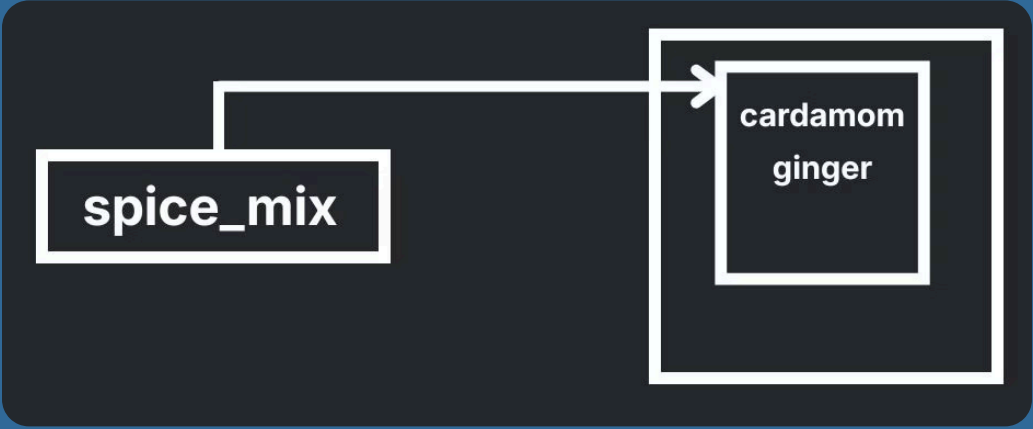
python:

```
spice_mix.add("Ginger")
print(spice_mix)
print(f'After spice mix id: {id(spice_mix)}')
```

output:

{'Cardamom', 'Ginger'}

After spice mix id: 2139208714496



- Note:
- Numbers, Strings and Tuples were immutable types.
 - From here onwards we will be learning about mutable types.

List In Python

- Lists are **used to store multiple items in a single variable**.
- A list is a **collection** which is **ordered** and **changeable** and it is similar to **Array's** which we see in other programming languages like **C++, Java, etc.**
- List also **allows duplicate values**.
- List's are created using **square brackets ('['] ')**.
- **Eg:**

```
fruit_list = ['Apple','Banana','Cherry']  
print(f"fruit_list: {fruit_list}")
```

Output: fruit_list: ['Apple', 'Banana', 'Cherry']

Some Important List Methods

1. **append()** : This method **appends an element to the end** of the list.

Eg:

```
grocery_list = ['Eggs', 'Milk', 'Cheese', 'Bread', 'Veggies']  
grocery_list.append('Meat')  
print(f"Grocery Items: {grocery_list}")
```

Output: Grocery Items: ['Eggs', 'Milk', 'Cheese', 'Bread', 'Veggies', 'Meat']

2. **remove()** : This method **removes the first occurrence of the element** with the specified value.

Eg:

```
grocery-list = ['Eggs', 'Milk', 'Cheese', 'Bread', 'Veggies', 'Meat', 'Cheese']  
grocery-list.remove('Cheese')  
print(f"Grocery Items: {grocery-list}")
```

Output: Grocery Items: ['Eggs', 'Milk', 'Bread', 'Veggies', 'Meat', 'Cheese']

3. **extend()** : This method **adds the specified list elements** (or any iterable) **to the end of the current list**.

Eg:

```
groceries = ['Bread', 'Milk']  
shopping = ['Jeans', 'Jacket']  
groceries.extend(shopping)  
print(f"Extended grocery list: {groceries}")
```

Output: Extended grocery list: ['Bread', 'Milk', 'Jeans', 'Jacket']

List In Python

4. **insert()** : This method **inserts the specified value at the specified position**.

Eg:

```
homemade_spices = ["Red Chili Powder", "Mango Pickle", "Lemon Pickle"]

homemade_spices.insert(1,"Turmeric Powder")

print(f'home made spices: {homemade_spices}')
```

Output: home made spices: ['Red Chili Powder', 'Turmeric Powder', 'Mango Pickle', 'Lemon Pickle']

5. **pop()** : This method **removes the element at the specified position**.

Eg:

```
homemade_spices = ["Red Chili Powder", "Turmeric Powder", "Mango Pickle", "Lemon Pickle"]

homemade_spices.pop(1)

print(homemade_spices)
```

Output: ["Red Chili Powder", "Mango Pickle", "Lemon Pickle"]

6. **reverse()** : This method **reverses the sorting order of the elements**.

Eg:

```
numlist = [26, 19, 12, 96, 74]
print(f"Num list: {numlist}")
```

Output: Num list: [26, 19, 12, 96, 74]

```
numlist.reverse()
print(f"Reversed Num list: {numlist}")
```

Output: Reversed Num list: [74, 96, 12, 19, 26]

7. **sort()** : This method **sorts** the list **ascending by default**.

Eg:

```
numlist = [26, 19, 12, 96, 74]
print(f"Num list: {numlist}")
```

Output: Num list: [26, 19, 12, 96, 74]

```
numlist.sort()
print(f"Sorted numlist: {numlist}")
```

Output: Sorted numlist: [12, 19, 26, 74, 96]

Sorting in Descending order

```
numlist.sort(reverse = True)
print(f"Reverse Sorted numlist: {numlist}")
```

Output: Reverse Sorted numlist: [96, 74, 26, 19, 12]

List In Python

8. **max()** : This method **fetches the maximum number** from the list

```
numlist = [26, 19, 12, 96, 74]
print(max(numlist))
```

Output: 96

9. **min()** : This method **fetches the minimum number** from the list

```
numlist = [26, 19, 12, 96, 74]
print(min(numlist))
```

Output: 12

Operator Overloading

- The feature in Python, which **allows the same operator** to **have different meanings depending on the context** is called **operator overloading**.
- As we know, the **+ operator** can perform addition on two numbers, merge two lists, or concatenate two strings.
- With some tweaks, we can use the **+ operator** to work with user-defined objects as well.

Eg:

```
base_item = ['Milk','Cone']
extra_flav = ['Vanilla']

ice_cream = base_item + extra_flav

print(f"ice cream: {ice_cream}")
```

Output: ice cream: ['Milk', 'Cone', 'Vanilla']

Bytearray function

- This function **convert objects into bytearray objects**, or **create empty bytearray object of the specified size**.
- It **returns a bytearray object**.

Eg:

```
animal_name = bytearray(b"Donkey")
animal_name = animal_name.replace(b"Don",b"Mon")
print(f"Bytes: {animal_name}")
```

Output: Bytes: bytearray(b'Monkey')

Sets In Python

- Sets are **used to store multiple items in a single variable**.
- A set is a collection which is **Unordered, Unchangeable** and **Unindexed**.
- Since it is unindexed, **duplicates are not allowed**.
- Sets are created using **curly braces ({ })**.

Eg:

```
fruits_set = {'Apple', 'Cherry', 'Peach', 'Banana'}
veggies_set = {'Cauliflower', 'Cabbage', 'Peas', 'Carrot'}

print(f"fruits_set: {fruits_set}")
print(f"veggies_set: {veggies_set}")
```

Output: fruits_set: {'Cherry', 'Banana', 'Peach', 'Apple'}

Output: veggies_set: {'Cabbage', 'Cauliflower', 'Peas', 'Carrot'}

Union in Set (|)

- It contains **all the items from both the sets**.

```
union = fruits_set | veggies_set
print(f"union is: {union}")
```

Output: union is: {'Cherry', 'Cabbage', 'Peas', 'Banana', 'Carrot', 'Peach', 'Apple', 'Cauliflower'}

Intersection in Set (&)

- It contains **similarity between the sets**.

```
fruits_set = {'Apple', 'Cherry', 'Tomatoe', 'Banana'}
veggies_set = {'Cauliflower', 'Cabbage', 'Peas', 'Tomatoe'}

inter = fruits_set & veggies_set
print(f"intersection: {inter}")
```

Output: intersection: {'Tomatoe'}

Difference in Set (-)

- It **removes** the **similar element from first set which also lies in second set**.

```
fruits_set = {'Apple', 'Cherry', 'Tomatoe', 'Banana'}
veggies_set = {'Cauliflower', 'Cabbage', 'Peas', 'Tomatoe'}
difference = fruits_set - veggies_set
print(f"difference is: {difference}")
```

Output: difference is: {'Banana', 'Apple', 'Cherry'}

Membership Test in Set

- It is used to **detect** whether a **following string or item lies inside a set or not**.
- It is done by using **in** and **not in** operator

```
accessories = {'watch', 'bracelet', 'Ring', 'Pendant'}
print('Ring' in accessories) // will print True
print('Chain' in accessories) // will print False
print('Chain' not in accessories) // will print True
print('watch' not in accessories) // will print False
```

Dictionary In Python

- Dictionary's are used to store data in "**key: value**" pair.
- A dictionary is a collection which is **Ordered, Changeable** and **Unindexed**.
- Since it is unindexed, **duplicates are not allowed**.
- Dictionaries are created using **curly braces ({ })** but unlike set's the data inside them are in **Key: value** pair format.
- **Eg:**

```
Dress={
'color':'blue',
'price': '₹1200/-',
'size': 'XL',
'tag': 'partywear'
}

print(f"Dress dictionary: {Dress}")
```

Output: Dress dictionary: {'color': 'blue', 'price': '₹1200/-', 'size': 'XL', 'tag': 'partywear'}

- Alternate way to create Dictionary

```
imgdict = {}
imgdict['image-type'] = 'Jpeg'
imgdict['img-size'] = '50Kb'
imgdict['img-date'] = '15-01-26'

print(f'image dictionary: {imgdict}')
```

Output: image dictionary: {'image-type': 'Jpeg', 'img-size': '50Kb', 'img-date': '15-01-26'}

Keys, Values and Items

- Returns **keys, values** and **items** of **any dictionary in the form of Lists**.

```
Dress={
'color':'blue',
'price': '₹1200/-',
'size': 'XL',
'tag': 'partywear'
}
print(Dress.keys())
print(Dress.values())
print(Dress.items())
```

Output:
dict_keys(['color', 'price', 'size', 'tag'])
dict_values(['blue', '₹1200/-', 'XL', 'partywear'])
dict_items([('color', 'blue'), ('price', '₹1200/-'), ('size', 'XL'), ('tag', 'partywear')])

Some useful dictionary methods

- **popitem()** : removes the last inserted item from the dictionary

```
Dress={
'color':'blue',
'price': '₹1200/-',
'size': 'XL',
'tag': 'partywear'
}

print(Dress)
```

Output: {'color': 'blue', 'price': '₹1200/-', 'size': 'XL', 'tag': 'partywear'}

```
Dress.popitem()
print(Dress)
```

Output: {'color': 'blue', 'price': '₹1200/-', 'size': 'XL'}

- **update()** : This method will **update the dictionary with the items from a given argument**.

```
imgdict = {}
imgdict['img-type'] = 'png'
imgdict['img-size'] = '200Kb'
imgdict['img-date'] = '16-01-26'

print(imgdict)
```

Output: {'img-type': 'png', 'img-size': '200kb', 'img-date': '16-01-26'}

```
imgdict.update({"img-type": "jpeg"})
print(imgdict)
```

Output: {'img-type': 'jpeg', 'img-size': '200kb', 'img-date': '16-01-26'}

If the item **does not exist**, the **item gets added**.

```
imgdict.update({"img-resolution": "4k"})
print(imgdict)
```

Output: {'img-type': 'jpeg', 'img-size': '200kb', 'img-date': '16-01-26', 'img-resolution': '4k'}

- **get()** : This method will **returns the value of the item with the specified key**.

```
imgdict.get('img-size','No size given')
```

Output: 200kb

```
imgdict.pop('img-size')
imgdict.get('img-size','No size given')
```

Output: No size given